Contents

AI-Assisted Recruitment Decision Support Platform

Business Requirements & Solution Design Document (v2)

Project type: Portfolio project (Business Analyst + AI Solution Design) **Author:** Sameer **Version:** 2.1 — implementation notes added (see Section 17 for changelog) **Scenario:** A mid-size company's recruitment team receives ~500 CVs per open role and needs to screen them efficiently, consistently, and fairly, without letting automation make hiring/rejection decisions on its own.

---

0. Problem Discovery

Before defining requirements, it's worth establishing that this problem is real, how it's currently handled, and what assumptions the proposed solution depends on.

Why this problem is real:

- High-volume recruitment screening is a well-documented bottleneck — recruiters report spending the majority of screening time on repetitive comparison against JD requirements rather than higher-judgment evaluation.
- Legacy ATS keyword-matching tools are widely known to misjudge strong candidates whose CVs don't follow a "scannable" format, effectively penalizing writing style rather than qualifications.
- Regulators have already responded: the EU AI Act classifies recruitment/CV-screening AI as a **high-risk AI system**, requiring human oversight, transparency, and documentation — signaling this isn't a hypothetical concern but an active compliance area.

How this is handled today (alternatives scan):

- **Manual screening** — accurate in principle, but slow and inconsistent between reviewers; also subject to human versions of the same biases (see below).
- **Legacy ATS keyword tools** — fast, but formatting-brittle and offer no explanation for why a candidate was filtered out.
- **General-purpose LLM chat tools used ad hoc** (a recruiter pasting a CV into ChatGPT) — flexible, but no structured extraction, no audit trail, no consistency between uses, no fairness safeguards built in.

None of these fully solve the combination of *speed* + *consistency* + *fairness* + *auditability* together — which is the gap this platform is designed to close.

Key assumptions this solution rests on (stated explicitly, since they're testable, not guaranteed):

- Recruiters will actually trust and act on explained recommendations rather than ignoring the tool or rubber-stamping it without review.
- A reasonably-sized synthetic/structured dataset is sufficient to demonstrate the mechanism, even though real-world CV diversity is larger.
- Structured LLM-based extraction is meaningfully more format-tolerant than legacy keyword ATS — this is a claim the project should demonstrate with real examples, not just assert.
- Fairness issues (formatting, demographic, order-of-arrival) are addressable through *process design* (routing rules, batch review) rather than requiring a fundamentally different model architecture.

---

1. Business Problem

Recruiters receiving hundreds of CVs per role spend the majority of their time on repetitive first-pass screening — reading each CV against the job description's requirements — rather than on higher-judgment work like interviewing and closing candidates. This is slow (screening a single high-volume role can take days), inconsistent (different recruiters apply requirements differently), and doesn't scale during hiring spikes.

Compounding this, purely automated keyword-matching tools (legacy ATS systems) are known to systematically underscore strong candidates whose CVs use non-standard formatting (tables, multi-column layouts, unconventional section headers) — meaning naive automation risks making the fairness problem worse, not better, if adopted without safeguards.

A separate, less-discussed bias also affects manual screening: **application-order bias**. Recruiters reviewing CVs as they arrive tend to give earlier applicants more attention and benefit of the doubt, while later applicants — even stronger ones — get reviewed under time pressure near the deadline, or after the recruiter has already mentally "filled" the role with early candidates. A candidate who is a 75% match but applies the day before the deadline can lose out to a 70% match who applied on day one, purely due to timing, not merit.

Core problem: Recruitment teams need to screen CV volume faster and more consistently, without introducing new bias (formatting-driven or otherwise) or removing human judgment from the actual hiring decision.

2. Stakeholders

Stakeholder	Interest / Concern
HR Manager	Wants screening time reduced significantly (e.g. “cut time-to-shortlist by 60%”) without adding headcount
Recruiter	Wants the top candidates surfaced fast, with a clear reason for every recommendation — needs to trust the tool enough to act on it
Hiring Manager	Wants a short, high-quality list of candidates worth interviewing, not a raw ranked dump
Candidate	Wants a fair shot — not to be silently filtered out by an algorithm they can’t see or appeal, especially due to CV formatting rather than qualifications
AI/ML Team	Owns model behavior, accuracy, and drift; concerned about maintainability and about being blamed for opaque failures
Engineering Team	Owns the pipeline and integration; concerned about scope creep and reliability
Legal/Compliance	Concerned about discriminatory outcomes and regulatory exposure (e.g. EU AI Act classifies recruitment AI as high-risk, requiring human oversight and transparency)

These stakeholders don’t fully agree — the HR Manager’s speed goal, the Hiring Manager’s quality goal, the Recruiter’s trust/explainability goal, and the Candidate’s fairness goal are in some tension, and the design has to hold all four at once rather than optimizing for just one.

3. As-Is Process

1. Recruiter posts a job description and receives CVs via an ATS or email.
2. Recruiter (or a junior team member) opens each CV individually and manually checks it against the JD’s requirements.
3. Candidates are informally sorted into “yes / maybe / no” piles based on individual reviewer judgment — no structured scoring.
4. Strong “maybe” candidates may get deprioritized simply because their CV is dense, unconventionally formatted, or uses different terminology than the JD.
5. Shortlist is handed to the hiring manager, often with little written justification for why each candidate made the cut.
6. Rejected candidates receive no meaningful feedback; there’s no record of why a candidate was screened out.

4. Pain Points

- **Volume vs. time:** manual review of hundreds of CVs per role is slow and doesn’t scale during hiring spikes.
- **Inconsistency:** two recruiters can reach different conclusions on the same CV, with no shared standard.
- **No structured trail:** decisions aren’t documented, making it hard to audit or explain outcomes later.
- **Formatting bias risk:** both human skimming and naive automated keyword tools can undervalue strong candidates whose CVs aren’t written in a “scannable” style — a well-qualified candidate can be missed for reasons unrelated to their qualifications.

- **No visibility for the candidate or company** into why a decision was made, which is a growing regulatory as well as ethical concern.

5. Proposed AI Solution

An **AI-Assisted Recruitment Decision Support Platform** that ranks and explains — but never autonomously rejects.

Pipeline: **Job Description → extract structured requirements → CV → extract structured candidate info → match candidate against requirements → generate explainable score → recruiter reviews → shortlist / interview decision (human-made).**

Two design principles anchor the whole system:

1. **The AI must assist, not autonomously decide.** No candidate is auto-rejected. Every output is a ranked recommendation with a visible explanation; a human makes the actual shortlist/reject decision.
2. **The system must not penalize candidates for CV formatting.** This is addressed directly via a dual-signal design (see below) rather than left as an unaddressed risk.
3. **The system must not penalize candidates for when they applied.** Ranking is computed only after the full applicant batch (or a defined cutoff) is in, and is based purely on match score — not on submission order or timestamp. A strong candidate who applies just before the deadline is ranked exactly as they would be if they'd applied on day one.
4. **The system must never expose or filter on protected characteristics.** All filters and matching logic operate only on merit-based fields — skills, experience, education, employer history, projects, extracurriculars, referral source. No filter is ever built on gender, age, ethnicity, religion, or any other protected characteristic, consistent with the platform's core fairness commitment.

Solving the ATS-formatting fairness problem

Rather than producing a single match score, the system computes **two independent signals** for every CV:

- **Extraction confidence** — how much structured information could actually be pulled from this CV (fields found vs. expected, ratio of structured content to raw CV length, whether fields came from clearly labeled sections or had to be inferred).
- **Match score** — fit between the JD's requirements and whatever candidate information *was* successfully extracted.

Routing logic:

- High extraction confidence + any match score → ranked normally in the main list.
- **Low extraction confidence, regardless of match score** → routed to a separate "Needs Manual Review" bucket with a note explaining why (e.g. "Only 2 of 6 expected fields extracted — CV formatting may have limited automated parsing"). This candidate is never silently ranked low.
- As a backstop, raw-text semantic similarity (whole JD vs. whole CV text, independent of field extraction) is also computed; if it's meaningfully higher than the structured match score suggests, that reinforces the review flag.

This turns "the AI might penalize non-standard CVs" from an acknowledged risk into a designed, testable, demoable safeguard.

Solving application-order bias

The platform processes CVs in **batch, not in arrival order**. It does not rank or surface candidates as they trickle in — it waits until the full applicant pool is in (or a defined review cutoff), then computes match scores for the entire batch together and ranks purely by score. This means:

- A candidate applying on day one and a candidate applying an hour before the deadline are scored identically, using the same JD, the same extraction logic, the same match calculation — timing plays no role in the score.
- The recruiter's view is always the full ranked batch, not a rolling list they've already started mentally committing to before the deadline — removing the "early candidates get more benefit of the doubt" dynamic that affects manual review.
- Submission timestamp is logged for audit purposes only (e.g. confirming the candidate applied within the window) — it is never a scoring input.

6. Functional Requirements

ID	Requirement
FR-1	System shall accept a job description as input and extract structured requirements (must-have skills, experience level, education, nice-to-haves)
FR-2	System shall accept a CV (text or PDF) and extract structured candidate information (skills, years of experience, education, past roles)
FR-3	System shall compute a match score between extracted candidate information and JD requirements
FR-4	System shall compute an extraction confidence score per CV, independent of match score
FR-5	System shall route any CV with extraction confidence below a defined threshold to a "Needs Manual Review" bucket, regardless of its match score
FR-6	System shall compute a raw-text semantic similarity score as a backstop signal alongside structured matching
FR-7	System shall generate a grounded explanation for every score, based on extracted facts (e.g. "matched 4/6 required skills, missing X, meets experience threshold"), not free-form LLM narration
FR-8	System shall present ranked candidates, flagged low-confidence candidates, and explanations in a recruiter-facing review interface
FR-9	System shall never auto-reject a candidate; all candidates remain visible to the recruiter, including low-ranked ones

ID	Requirement
FR-10	System shall log every JD, CV, extracted fields, scores, routing decision, and recruiter action for later audit and reporting
FR-11	System shall support adding new JDs and CVs without a code change
FR-12	System shall expose the JD’s extracted required skills as interactive filters, allowing recruiters to filter the candidate list by one or more required skills
FR-13	System shall compute skill-fit and experience-fit as separate sub-scores, and allow the recruiter to adjust their relative weighting (e.g. “skills-first”, “balanced”, “experience-first”) to re-rank candidates according to hiring priority
FR-14	System shall process candidates in batch (all CVs received within the review window) rather than in arrival order, and shall rank purely by computed match score, never by submission timestamp
FR-15	System shall log each candidate’s submission timestamp for audit purposes only — it must never be used as a scoring input or influence ranking
FR-16	System shall support an on-demand agentic explanation workflow: when a recruiter asks why a specific candidate needs review, the system shall chain together retrieval of the candidate’s extracted information, an extraction-confidence check, a missing-requirements check, retrieval of relevant recruitment policy, and a grounded recommendation (Review / Proceed / Need More Information)
FR-17	System shall perform deep project-level analysis for each candidate: identify projects mentioned in the CV, assess relevance to the JD’s domain/tech stack, and generate a short summary with extracted keywords for each relevant project
FR-18	System shall treat known synonyms and equivalent terms (e.g. “React” / “ReactJS” / “React.js”) as matching during skill comparison, so candidates are not penalized for wording differences alone
FR-19	System shall allow the recruiter to filter and sort candidates by years of experience, academic institution, previous employer, specific skills, relevant projects, extracurricular activities, and referral source

ID	Requirement
FR-20	System shall present candidates in two segments — a main list and a “Needs Manual Review” list — accessible via a single tap/toggle, with both kept equally visually prominent
FR-21	System shall provide a per-candidate action to download or open the original CV PDF at any time, for manual verification
FR-22	System shall allow the recruiter to add a free-text note to any candidate profile, stored alongside (never overwriting) the AI-generated explanation
FR-23	System shall provide a free-text search across extracted candidate data (skills, projects, past roles), in addition to structured filters
FR-24	System shall allow the recruiter to select multiple candidates and shortlist them in a single bulk action
FR-25	System shall allow the recruiter to export the current shortlist as a PDF or Excel file
FR-26	System shall provide a hiring tracker view showing candidates grouped by pipeline stage (Shortlisted, Interview, Hired)
FR-27	System shall display each tracker candidate’s phone, email, and address alongside their stage, sourced from the same extracted data as the main list

7. Non-Functional Requirements

ID	Requirement
NFR-1	Screening a batch of CVs against one JD should complete in a reasonable batch-processing time (not required to be real-time)
NFR-2	The system must clearly distinguish AI-generated recommendations from recruiter-approved decisions in any view
NFR-3	The system must not process or store real candidate PII — portfolio version uses synthetic/anonymized CV data only, as a deliberate privacy-by-design decision
NFR-4	Every score and routing decision must be traceable to the specific extracted fields and rule that produced it (auditability)
NFR-5	The system must degrade toward human review rather than toward a confident-looking guess whenever extraction confidence is low

ID	Requirement
NFR-6	UI must be usable by recruiters with no AI/technical background
NFR-7	The system's design must reflect awareness of high-risk AI system obligations for recruitment tools under frameworks like the EU AI Act (human oversight, transparency, documentation)

8. User Stories

- *As a recruiter*, I want ranked candidates with a clear explanation for each score, so I can trust and act on the recommendation quickly.
- *As a recruiter*, I want candidates whose CVs couldn't be reliably parsed flagged separately, so I don't accidentally overlook a strong candidate due to formatting.
- *As a hiring manager*, I want a short, high-quality shortlist rather than a raw ranked dump of all 500 candidates.
- *As an HR manager*, I want visibility into how much screening time is being saved, so I can justify the tool's continued use.
- *As a candidate*, I want assurance that a formatting quirk in my CV won't cause me to be silently filtered out without human review.
- *As a compliance stakeholder*, I want every recommendation traceable to extracted facts and a documented rule, so decisions can be audited and defended.
- *As an AI/ML team member*, I want extraction confidence tracked separately from match score, so parsing failures aren't mistaken for weak candidates.
- *As a recruiter*, I want to filter candidates by specific required skills, so I can quickly find who has a must-have qualification even if their overall rank is lower.
- *As a recruiter*, I want to prioritize either skill match or experience level depending on hiring urgency, so the ranking reflects what actually matters for this specific role right now (e.g. a strong-but-junior candidate for a skills-critical role vs. an experienced candidate needed to staff a project immediately).
- *As a candidate*, I want to know that applying close to the deadline won't disadvantage me compared to someone who applied earlier with a weaker fit, so I'm judged on merit, not timing.
- *As a recruiter*, I want to ask why a specific candidate needs manual review, so I get a clear, traceable justification instead of digging through raw extracted data myself.
- *As a recruiter*, I want the AI to go beyond a skills checklist and analyze a candidate's past projects in depth, so I can see real applied experience, not just keyword overlap.
- *As a recruiter*, I want skill matching to recognize synonyms and equivalent terms, so qualified candidates aren't missed purely due to wording differences.
- *As a recruiter*, I want to filter and sort candidates by experience, academic institution, previous employer, skills, projects, extracurriculars, and referral source, so I can quickly find candidates matching very specific hiring needs (e.g. "previously worked at bKash").
- *As a recruiter*, I want to switch between the main candidate list and the review-flagged list with one tap, so both groups stay equally accessible and neither is out of sight.
- *As a recruiter*, I want to download any candidate's original CV at any time, so I can manually verify anything the AI extracted.
- *As a recruiter*, I want to leave a note on a candidate's profile, so my own observations travel alongside the AI's explanation for future reference.
- *As a recruiter*, I want to search candidates by keyword, so I can quickly locate anyone matching a specific term not covered by structured filters.
- *As a recruiter*, I want to shortlist multiple candidates at once, so I don't have to repeat the same action one by one.

- *As a recruiter*, I want to export my shortlist as a PDF or Excel file, so I can hand it directly to the hiring manager.

9. Acceptance Criteria

For "recruiter gets an explainable ranked list":

- Given a batch of CVs and a JD, when processing completes, then each CV shows a match score and an explanation grounded in specific extracted fields (not a generic LLM paragraph).

For "the AI never autonomously rejects":

- Given any CV, when processed, then it remains visible to the recruiter in some view (ranked list or review bucket) — no CV is silently dropped from the output entirely.

For "formatting doesn't silently penalize a candidate":

- Given a CV where extraction confidence falls below the defined threshold, when processed, then the candidate is placed in the "Needs Manual Review" bucket with an explanation of which fields could not be reliably extracted, regardless of its computed match score.
- Given two CVs with identical underlying qualifications but different formatting (one standard, one non-standard/table-heavy), when both are processed, then the non-standard one is not silently ranked lower without a review flag explaining reduced extraction confidence.

For "skill filtering doesn't undo the fairness safeguard":

- Given a recruiter filters the candidate list by a required skill, when the filter is applied, then any "Needs Manual Review" candidates remain visible in the filtered view (not hidden), shown with a note that the skill could not be confidently confirmed from their CV.

For "weighting reflects hiring priority":

- Given a recruiter selects "skills-first" weighting, when the list re-ranks, then a candidate with a higher skill-fit sub-score but lower experience-fit sub-score ranks above a candidate with the reverse profile (and vice versa for "experience-first").
- Given any weighting is selected, when the list re-ranks, then "Needs Manual Review" candidates are not blended into the ranked list — they remain in their own bucket regardless of weighting, since their sub-scores may be unreliable.

For "application timing doesn't affect ranking":

- Given two candidates with different submission timestamps but the system has reached its review cutoff, when the batch is ranked, then a candidate with a higher match score ranks above a candidate with a lower match score, regardless of which one applied earlier.
- Given the candidate list is displayed, when a recruiter inspects the ranking logic or audit log, then submission timestamp appears only as metadata and is never cited as a factor in the score or explanation.

For "agentic explanation workflow":

- Given a recruiter asks why a specific candidate needs manual review, when the workflow runs, then it returns, in order: the candidate's extracted information, their extraction confidence, which requirements are missing (if any), any relevant policy note, and a final recommendation of Review / Proceed / Need More Information.
- Given the workflow cannot retrieve one of its inputs (e.g. no policy note applies), when it runs, then it states that explicitly rather than fabricating a plausible-sounding but ungrounded answer.

For "auditability":

- Given any candidate's score in the log, when reviewed, then the specific extracted fields, the rule applied, and the routing decision can all be traced.

For "deep project analysis":

- Given a CV containing project descriptions, when processed, then each relevant project is summarized with extracted keywords and a stated relevance to the JD, rather than only being counted toward a generic skill match.

For "synonym-aware matching":

- Given a CV lists an equivalent term for a required skill (e.g. "ReactJS" for "React"), when matched against the JD, then the skill is counted as met, not missed.

For "extended filtering":

- Given a recruiter filters by a specific previous employer, when applied, then only candidates whose extracted work history includes that employer are shown.
- Given a recruiter searches a keyword, when results return, then matches from skills, projects, and past roles are all included, not just top-level fields.

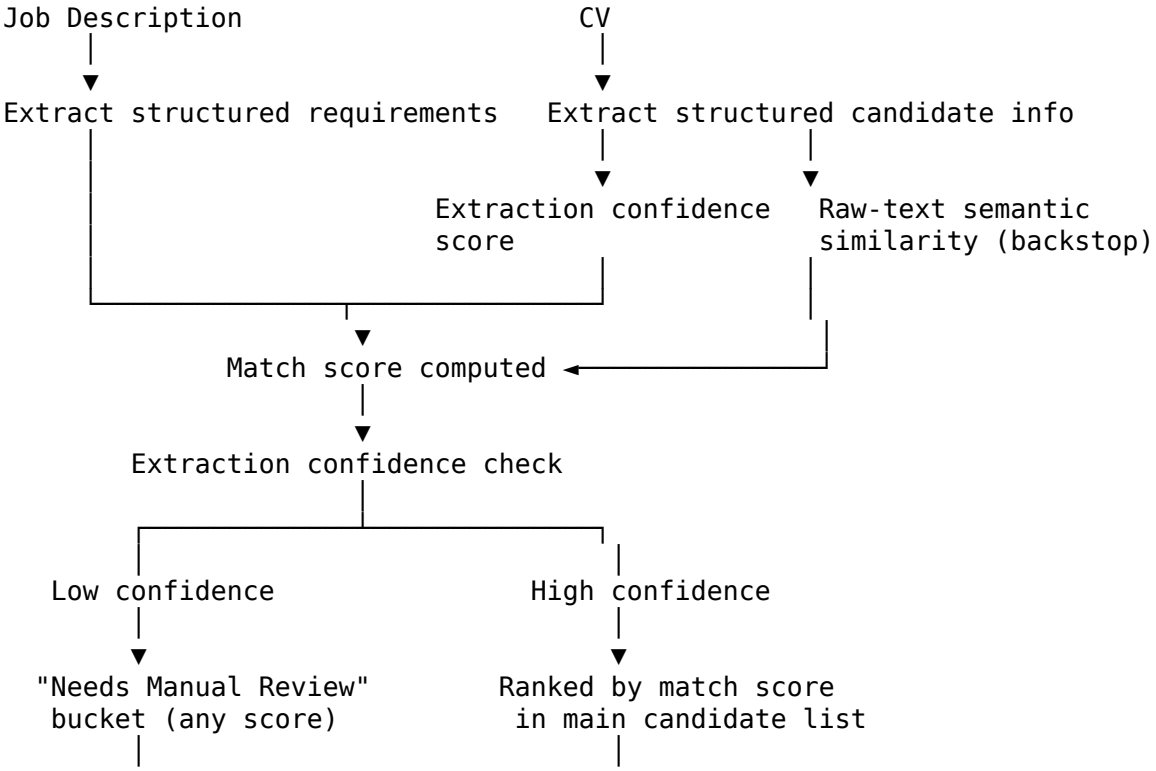
For "two-segment interface":

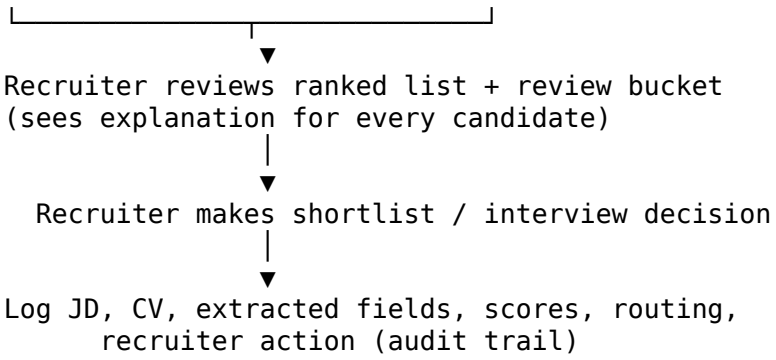
- Given a recruiter taps the "Needs Manual Review" segment, when viewed, then all flagged candidates are shown with equal visual weight to the main list — never as a smaller or secondary-looking section.

For "no protected-characteristic filtering":

- Given the full filter and sort option list, when reviewed, then no option is based on gender, age, ethnicity, religion, or any other protected characteristic — only merit-based fields are present.

10. Process Workflow





11. MVP Scope

In scope for MVP:

- One JD parsed into structured requirements
- A synthetic/anonymized CV dataset (mix of standard-formatted and deliberately non-standard-formatted CVs, including 2-3 strong-but-oddly-formatted candidates for demo purposes)
- Structured CV information extraction
- Match scoring (TF-IDF or embedding-based similarity, consistent with prior project's approach)
- Extraction confidence scoring and the low-confidence review-routing rule
- Raw-text semantic similarity backstop
- Grounded, fact-based explanation generation
- On-demand agentic explanation workflow for individual candidate review queries
- Basic Streamlit recruiter UI: ranked list, review bucket, explanations, approve/shortlist action
- Logging to local file/CSV for audit and reporting
- Three small controlled fairness experiments (10-20 synthetic CVs each): formatting-bias, demographic swap-test, order-independence check — documented findings, not large-scale statistical studies

Nice-to-have, not required for MVP completion:

- Public deployment via Streamlit Community Cloud — valuable if time allows, but a working local/Colab MVP with clean GitHub code and a strong README is the priority over a deployed link. Time is better spent improving the actual solution than debugging deployment issues.

v2 additions — designed (documented, in requirements/wireframe) but not all built into the working demo:

- Deep project-level analysis, synonym-aware matching, two-segment interface, and the extended filter set (experience, institution, employer, skills, projects, extracurriculars, referral) are fully specified and shown in the wireframe.
- For the working prototype, only a subset is actually implemented end-to-end (e.g. keyword search and the two-segment toggle) — the rest are demonstrated through the wireframe and written requirements rather than working code, given the 2-day build window. This is a deliberate scope decision, not an oversight.

Explicitly out of scope for MVP (documented as backlog):

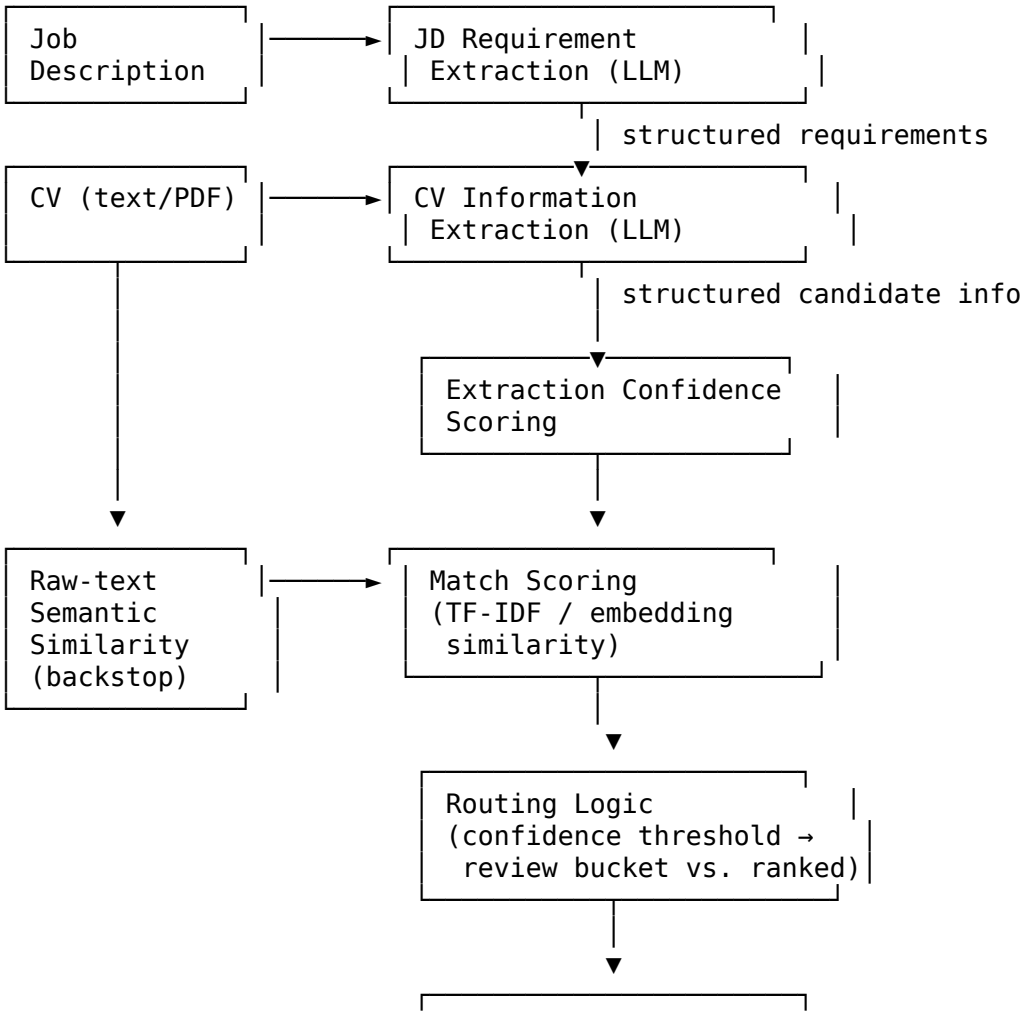
- Real ATS integration (e.g. Greenhouse, Lever)
- Large-scale statistical bias auditing across many demographic variables (MVP uses small controlled swap-tests instead, ~10-20 CVs per test)

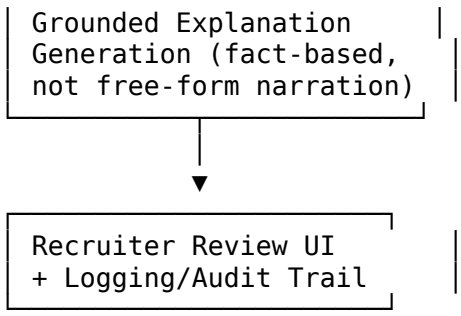
- Multi-role/multi-JD batch comparison
- Candidate-facing transparency portal (e.g. showing candidates their own explanation)
- Authentication/role-based access
- Side-by-side comparison of 2-3 candidates

12. Product Backlog (Post-MVP)

1. Integrate with a real ATS (Greenhouse/Lever) instead of a standalone UI
2. Expand bias auditing beyond name/university swap-test to a larger, statistically-designed fairness evaluation
3. Add a candidate-facing transparency view showing why they were or weren't shortlisted
4. Add recruiter feedback loop (override tracking) to monitor and improve matching quality over time
5. Support comparing candidates across multiple concurrent JDs/roles
6. Add role-based access (recruiter vs. hiring manager vs. HR manager views)
7. Build a full analytics dashboard (time saved, review-bucket rate, override rate, per-role trends)
8. Add multi-language JD/CV support

13. AI Architecture





Agentic explanation workflow (on-demand, per candidate)

Separate from the main batch pipeline above, a small chained workflow runs on-demand when a recruiter asks a question like *"Why should I manually review Candidate 7?"*:

```
Recruiter question ("Why review Candidate 7?")
        │
        ▼
Step 1: Retrieve candidate's extracted information (from pipeline output)
        │
        ▼
Step 2: Check extraction confidence score
        │
        ▼
Step 3: Check which required fields/skills are missing
        │
        ▼
Step 4: Retrieve relevant recruitment policy note (e.g. review-threshold rule)
        │
        ▼
Step 5: Generate grounded explanation from steps 1-4 (fact-based, not free narration)
        │
        ▼
Step 6: Output recommendation — Review / Proceed / Need More Information
```

This is a genuine agentic pattern — a chain of retrieval and reasoning steps toward a decision — deliberately kept small and scoped to one clear use case, rather than expanding the whole platform into a general-purpose autonomous agent.

14. Risks / Limitations

Risk	Mitigation
AI penalizes candidates with non-standard CV formatting	Dual-signal design: extraction confidence tracked separately from match score; low confidence always routes to manual review regardless of score
Demographic bias (name, university, gendered language) affects scoring	Manual swap-test on a small CV set (identical content, varied name/university) as a designed fairness check; documented as a backlog item to scale up

Risk	Mitigation
Application-order bias — earlier applicants get more benefit of the doubt than later-but-stronger ones	Batch processing model: all candidates within the review window are scored and ranked together, purely by match score; submission timestamp is logged for audit only, never used in scoring
Recruiter over-trusts AI ranking and stops reviewing review-bucket candidates	UI design keeps review bucket equally visible, not buried; explicit acceptance criterion that no candidate is silently dropped
Explanation is plausible-sounding but not actually grounded in facts	Explanation generation is fact-based (built from extracted fields), not open-ended LLM narration
Small demo dataset doesn't reflect real-world CV diversity	Explicitly scoped as MVP; backlog covers larger-scale evaluation
Regulatory exposure (recruitment AI is high-risk under frameworks like the EU AI Act)	Human-in-the-loop is a hard requirement, not a nice-to-have; every decision is auditable and traceable
Candidate privacy	Synthetic/anonymized CVs only; no real candidate PII used or stored

15. Success Metrics

- **Screening time reduction:** illustrative estimate of time-to-shortlist reduction vs. as-is manual process (clearly labeled as a projected/simulated figure, not a claimed production result)
- **Review-bucket rate:** % of CVs routed to manual review due to low extraction confidence — tracked over time to see if it drops as parsing improves, and to confirm it isn't being ignored by recruiters
- **Ranking consistency:** whether the same CV/JD pair produces a stable score across repeated runs
- **Explanation groundedness:** qualitative check that explanations reference only facts actually present in the extracted data, not fabricated details
- **Bias swap-test outcome:** whether identical CV content with varied name/university produces materially different scores (target: no significant difference)
- **Order-independence check:** whether identical CV content produces the same score and rank regardless of a simulated earlier or later submission timestamp (target: no difference)
- **Recruiter override rate:** how often a recruiter's final decision disagrees with the AI's ranking (expected and healthy to be non-zero — this is a signal to monitor, not a failure metric)

16. Implementation Notes (Prototype Build)

Once the working Streamlit prototype was built, a few things became clearer than they were on paper:

What's real vs. UI-only in the demo: - **Fully working:** structured candidate ranking (always sorted by match score), the two-segment interface, search, the agentic explanation workflow (a real call to an open-weight LLM), the hiring tracker (FR-26/27), and toggling an explanation open/closed - **Designed and shown in the UI, but not wired to real logic:** all 5 filter pills (Skills, Experience, Institution, Previous employer, Projects) are cosmetic — selecting any of them does not change the results. The "Download CV" / "View original CV" buttons, the "Schedule interview / Reject / Compare with others" actions, and the tracker's "Move back a

stage" action are placeholders with no file or logic behind them. The "Run screening" button on the upload page does not read the uploaded JD text or CV files at all — it always displays the same pre-loaded mock candidates regardless of what was uploaded. This was a deliberate scope decision to protect time for the features that actually demonstrate the platform's core claims, not an oversight — and is disclosed here rather than left implicit.

Model choice: the agentic explanation feature uses an open-weight LLM (`openai/gpt-oss-20b`) served via Groq, chosen for cost (free tier), speed (fast enough for a live demo), and openness (an open-weight model is more inspectable than a fully closed one, which matters for the platform's governance story). For a production system, this choice would be re-evaluated against closed frontier models specifically on explanation-quality benchmarks.

A real bug worth documenting: the chosen model is a "reasoning" model — it spends part of its token budget on internal reasoning before writing the actual answer. Under a low token limit, harder candidates occasionally exhausted the entire budget on reasoning, returning a completely empty response with no error. Fixed by lowering the reasoning-effort setting (this task doesn't need deep reasoning) and raising the token budget, with a fallback message if it ever happens again. This is the kind of subtle AI-integration failure mode a PO should be able to reason about, even without writing the fix personally.

---

17. Version Notes

v1 → v2 changes:

- Added deep project-level analysis (FR-17) — matching goes beyond a skills checklist into whether a candidate's actual project work is relevant to the role.
- Added synonym-aware skill matching (FR-18) — closes a fairness gap where wording differences (e.g. "React" vs "ReactJS") could cause a qualified candidate to be undercounted.
- Expanded filtering and sorting (FR-19) — added institution, previous employer, projects, extracurriculars, and referral source, alongside the original skill-based filter.
- Formalized the two-segment interface (FR-20) as an explicit requirement, rather than leaving it implicit in the routing logic.
- Added per-candidate CV download (FR-21), recruiter notes (FR-22), search (FR-23), bulk shortlist (FR-24), and shortlist export (FR-25) — practical recruiter workflow features identified through further requirements review.
- Added an explicit design principle and acceptance criterion (Section 5, Section 9) stating that no filter or matching logic may be based on protected characteristics — a direct consequence of the platform's existing fairness commitment, made explicit rather than assumed.
- Deferred side-by-side candidate comparison to backlog (Section 12) to protect MVP scope.

v2 → prototype build (this update):

- Added FR-26 and FR-27 for the hiring tracker, which was built as working functionality during implementation, not just designed.
- Added Section 16 documenting what's real vs. UI-only in the demo, and the model-selection and debugging notes from the actual build.

This document is written as a real internal BA deliverable would be. The company and CV/JD examples are illustrative or synthetic; the design decisions (dual-signal scoring, human-in-the-loop routing, grounded explanations) are meant to be genuinely implementable, not just described.